BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Property true after processing each step.

Q2. Expected outcome after valid execution.

Q3. Defines valid types, conditions and outputs.

Q4. [3,10,7], sum20.

You Do pages · Independent questions

Q1. After40 count1 [2], after41 count2 [1], after39 still2 [1], after100 count3 [1], returns3 [1].

Q2. int count=0 [1], foreach array [1], if(value<0) [2], count++ [1], return count [1].

Q3. Count equals passes in examined prefix [2]; O(n) time [1], O(1) auxiliary memory [1], boundary39/40 and empty input [2].

Exit Ticket · Model answer

False. Preconditions shape correctness.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

LESSON 072 · Lesson 072 · Algorithm Design: From Specification to Steps

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What is a loop invariant?

Q2. What is a postcondition?

Q3. How is an input specification useful?

Q4. C# array a=[3,5,7]: set a[1]=a[0]+a[2]. Give final array and sum.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

algorithm — Finite unambiguous sequence of steps.

invariant — Property preserved across loop iterations.

precondition — Required input assumption.

postcondition — Expected condition after executing.

New learning

Observe — Read the problem to identify exact input type, required output, boundaries and invalid cases.

Explain — Write an algorithm with an invariant: after each iteration, count is the number of qualifying values examined so far.

Evaluate — Choose meaningful tests around the 40 boundary and compare output against an independently calculated result.

Worked example

Design a function counting even numbers in [1,2,6,7].

1. Explain — Using modulo2==0 identifies even values; the list has 2 and 6.

We Do · Practise with guidance

Design a function counting temperatures below zero in [−2,5,−1,0].

```
If x<0 increment count; −2 and −1 qualify → result2.
```

Explain

Zero does not count as negative, so use <0, not <=0.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Trace CountPasses for [40,41,39,100]. Show count after each item. [6]

Question 2

Write C# method that counts negative integers in an array, and returns0 for empty. [6]

Question 3

Explain a loop invariant, time complexity and edge tests for CountPasses. [6]

Exit Ticket · Answer before checking

An algorithm can be evaluated without considering its input requirements.